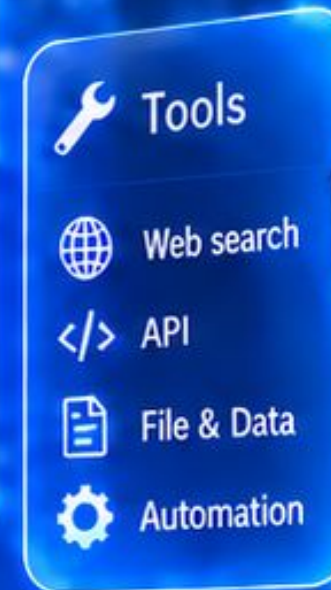


HERMES AGENT

BUILD YOUR OWN AI EMPLOYEE

จากผู้ใช้ AI → สู่ผู้สร้าง AI Agent



Agent



Memory



Tools



Skills



Workflow



Multi-Agent



AI Employee

ลงมือทำจริง • สร้างใช้กับงานของคุณ

AI
Productivity
Future Work



AI
BETTER
WORK

อ.เฟิร์ส วัชร น้อยศรีพันธ์

Agent Workflow & Multi-Agent

สร้างทีม AI

ให้ Agent วางแผน แตกงาน ตรวจสอบ และส่งงานกันเองได้



งานใหญ่ไม่ได้ต้องการ Agent ที่ฉลาดขึ้น แต่ต้องการลำดับงานที่ชัดเจน

กำหนดการวันนี้



เป้าหมายของวันนี้

01 ให้ Agent วางแผนและแตกงานใหญ่ออกเป็นขั้นตอนที่ทำได้จริง
แตกงานอย่างไรให้แต่ละขั้นมีผลลัพธ์ที่จับต้องได้

02 ออกแบบ Workflow ที่ตรวจได้ทุกขั้น ไม่ใช่สั่งรวดเดียวแล้วภาวนา
ใส่ Error Handling, Retry และ Verification ตรงไหนบ้าง

03 รู้ว่าเมื่อไหร่ควรใช้ Agent ตัวเดียว และเมื่อไหร่ควรแบ่งเป็นทีม
เส้นแบ่งระหว่าง “งานเดียวหลายขั้น” กับ “หลายบทบาท”

04 วาง Architecture ของทีม AI ที่จะใช้ต่อในโปรเจกต์วันที่ 5
Orchestrator ส่งงานให้ Sub-agent ด้วยอะไร และรับอะไรกลับ

DAY 4

Agent Workflow & Multi-Agent

จบวันนี้ Agent ของเรา
จะทำงานหลายขั้นจบเองได้
และมีตัวตรวจงานคนละตัวกับตัวทำ

ก่อนเริ่ม · เตรียมให้พร้อมใน 3 นาที

วันนี้ลงมือทำ 60 นาที จากทั้งหมด 120 — เวลาเตรียมเครื่องคือเวลาที่หายไปจาก Workshop

- 01

สร้างโฟลเดอร์ day4-workshop

ให้ Agent ทำงานอยู่ในโฟลเดอร์นี้เท่านั้น เหมือนที่ทำไว้เมื่อวันที่ 3
- 02

เอา Skill ของวันที่ 3 มาวางไว้

Workflow วันนี้จะเรียกใช้ Skill ที่แต่ละคนเขียนไว้เป็นหนึ่งขั้นตอน
- 03

เปิด Memory จากวันที่ 2 ไว้

User Memory และ Project Memory ทำให้ Agent ไม่ต้องถามบริบทเดิมซ้ำทุกชั้น
- 04

เลือกงานใหญ่หนึ่งชิ้นมาในใจ

งานที่ทำคนเดียวแล้วใช้เวลาเกินครึ่งวัน — นั่นคือโจทย์ของ Workshop 7

ติดตรงไหนให้พิมพ์ในแชทห้องได้เลย — ระหว่าง Workshop มีผู้ช่วยคอยตอบ ไม่ต้องรอนจบ

OI

Planning & Agent Workflow

Session 7 · 20:00 – 20:30

ทำไมงานใหญ่ถึงง่ายกว่างานเล็ก

สังรัดเดียว

Agent ที่เดาทางเอง

โจทย์ยาวหนึ่งย่อหน้า แล้วปล่อยให้ Agent ตัดสินใจทุกอย่างเอง พอผิดตั้งแต่ขั้นที่สอง ทุกขั้นหลังจากนั้นก็ผิดตาม และเรารู้ตอนจบ

แตกเป็นขั้น

Agent ที่หยุดให้ตรวจได้

แต่ละขั้นมีผลลัพธ์ของตัวเองที่เปิดดูได้ ผิดตรงไหนเห็นตรงนั้น แก้เฉพาะขั้นนั้นแล้วเดินต่อ ไม่ต้องเริ่มใหม่ทั้งงาน

ความต่างที่ตรวจสอบได้		
รู้ว่าผิดตอนไหน	ตอนอ่านผลลัพธ์สุดท้าย	ตอนจบขั้นที่ผิด
แก้แล้วเสียเวลาเท่าไร	รันใหม่ทั้งงาน	รันใหม่เฉพาะขั้นนั้น
เอาไปทำซ้ำได้ไหม	ต้องนึกเองว่าเคยสั่งอะไร	มีลำดับขั้นเป็นไฟล์อยู่แล้ว

แผนที่ใช้ได้จริง มีสามส่วน

Agent วางแผนไม่เป็น ไม่ใช่เพราะโง่ — แต่เพราะเราไม่ได้บอกว่าอะไรคือ “เสร็จ”

GOAL	STEPS	DONE
ปลายทางคืออะไร ได้ไฟล์ <code>proposal.md</code> ที่ลูกค้าอ่านแล้วตัดสินใจได้ เขียนเป็นสิ่งที่เปิดดูได้ ไม่ใช่ความรู้สึก — “ครบถ้วน” ไม่ใช่เป้าหมาย	ต้องผ่านอะไรบ้าง - รวบรวมข้อมูลลูกค้า - วิเคราะห์ความต้องการ - ร่างข้อเสนอ - ตรวจตัวเลขและเงื่อนไข แต่ละขั้นต้องมีผลลัพธ์ของตัวเอง ที่ขั้นถัดไปหยิบไป ใช้ได้ทันที	รู้ได้ยังไงว่าเสร็จ ทุกตัวเลขมีที่มา ไม่มีหัวข้อไหนเว้นว่าง มีรายการสิ่งที่ยังไม่รู้ เกณฑ์ที่เถียงกันไม่ได้ ถ้าอ่านแล้วยังตีความได้สอง แบบ แปลว่ายังไม่ใช้เกณฑ์

เขียนสามส่วนนี้ก่อนพิมพ์คำสั่งแรก ใช้เวลา 2 นาที — ประหยัดการรันซ้ำได้ทั้งคืน

เดินตาม Workflow จริง หนึ่งงาน

โจทย์ · “ทำสรุปคู่แข่งสามรายให้ทีมขาย ภายในวันนี้”

ขั้น	Agent ทำอะไร	ผลลัพธ์ที่ต้องได้	ถ้าไม่ผ่านทำอะไร
01	สรุปโจทย์เป็นแผนย่อยแล้วรอเราตอบ ok	แผน 4-6 ข้อที่เราอ่านแล้วเห็นภาพ	แก้แผนตรงนี้ ยังไม่ให้เริ่มทำ
02	ค้นข้อมูลคู่แข่งทีละราย แล้วบันทึกแหล่งที่มา	ไฟล์ research/ รายละเอียด พร้อมลิงก์	ขาดรายไหน ให้รันซ้ำเฉพาะรายนั้น
03	เทียบราคาและจุดขาย แล้วตัดสิ่งที่ไม่เกี่ยว	ตารางเปรียบเทียบที่มีแต่ข้อมูลที่ใช้จริง	ถ้าทุกช่องเต็มหมด แปลว่ายังไม่ได้ตัด
04	ร่างสรุปหนึ่งหน้าสำหรับทีมขาย	ไฟล์ summary.md ที่อ่านจบใน 2 นาที	ยาวเกินให้สั่งตัด ไม่ใช่สั่งเขียนใหม่
05	ตรวจย้อนว่าทุกข้ออ้างมีที่มาในไฟล์ research	รายการข้อที่ตรวจแล้ว และข้อที่ยังไม่แน่ใจ	เจอข้อที่ไม่มีที่มา ให้ถอดออก

สังเกตขั้นที่ 1 — Agent หุ้ดรอเรา นั่นคือจุดที่ถูกที่สุดในการแก้ทั้ง Workflow

งานแบบนี้ แตกเป็นกี่ขั้น

ขั้นที่ดีคือขั้นที่มีผลลัพธ์เปิดดูได้ ไม่ใช่ขั้นที่ฟังดูเป็นระเบียบ

“ทำรายงานประจำเดือน”

4 ขั้น

ดึงข้อมูล → รวมยอด → เทียบเดือนก่อน → เขียนสรุป

“ตอบอีเมลลูกค้าฉบับนี้”

1 ขั้น

งานเดียวจบ อย่าแตกให้มันยุ่งกว่าเดิม

“ย้ายเอกสารทีมขึ้นระบบใหม่”

หลายขั้น + วนซ้ำ

สำรวจ → จัดหมวด → ย้ายทีละหมวด → ตรวจสอบทีละหมวด

“หาสาเหตุที่ยอดตก”

3 ขั้น

ตั้งสมมติฐาน → หาข้อมูลมาหักล้าง → สรุปเฉพาะที่เหลือรอด

“เขียนบทความจากงานวิจัย”

5 ขั้น

อ่าน → สกัดประเด็น → วางโครง → ร่าง → ตรวจสอบข้อเท็จจริง

“เช็กว่าเว็บล่มหรือเปล่า”

1 ขั้น

งานที่จบในคำสั่งเดียว ไม่ต้องมี Workflow

เกณฑ์ตัดสิน — ถ้าขั้นนั้นไม่มีอะไรให้เราเปิดดู มันไม่ใช่ขั้น มันคือคำอธิบาย

สิ่งเสาที่ทำให้ Workflow เชื่อถือได้

Error Handling

บอกล่วงหน้าว่าถ้าขั้นไหนล้ม ให้ทำอะไรต่อ

- ไฟล์หาย ให้บอกแล้วหยุด
- API ล่ม ให้ข้ามไปขั้นอื่นก่อน
- ข้อมูลไม่ครบ ให้ถามกลับ

Retry

ลองใหม่แบบมีเงื่อนไข ไม่ใช่วนซ้ำวิธีเดิมไปเรื่อย ๆ

- ลองได้ไม่เกิน 2 ครั้ง
- รอบสองต้องเปลี่ยนวิธี
- ครบแล้วยังไม่ได้ ให้รายงาน

Verification

ตรวจสอบผลลัพธ์ด้วยเกณฑ์ ก่อนส่งให้ขั้นถัดไป

- เทียบกับเกณฑ์ที่เขียนไว้
- สุ่มตรวจสอบย้อนกลับ 2 ข้อ
- ให้ Agent คนละตัวตรวจ

Human-in-the-loop

จุดที่คนต้องอนุมัติ ก่อนงานเดินข้ามไปจุดที่ย่อนยาก

- ก่อนส่งออกนอกทีม
- ก่อนเขียนทับของเดิม
- ก่อนใช้เงินหรือให้สัญญา

ใส่จุดหยุดไว้ตรงไหนบ้าง

Workflow ที่ไม่มีจุดหยุด คือการปล่อยให้ความผิดพลาดวิ่งไปจนสุดสาย แล้วค่อยมารู้ตอนจบ

01	หลังวางแผน ก่อนเริ่มลงมือ	จุดที่ถูกที่สุด — แก่หนึ่งบรรทัดแทนการรันใหม่ทั้งงาน
02	ก่อนการกระทำที่ย้อนกลับไม่ได้	ส่งอีเมล ลบไฟล์ จ่ายเงิน เปลี่ยนสถานะในระบบจริง
03	เมื่อ Agent เจอทางแยกที่ข้อมูลไม่พอ	ให้มันถามพร้อมตัวเลือก ไม่ใช่เลือกเองแล้วเขียนเหตุผลสวย ๆ
04	ก่อนส่งผลงานออกนอกทีม	คนเซ็นชื่อรับผิดชอบยังเป็นคน — Agent ร่างได้ แต่ไม่ได้อนุมัติ

จุดหยุดที่ดีใช้เวลาคนไม่เกิน 30 วินาทีต่อครั้ง — ถ้านานกว่านั้น แปลว่าเราขอให้คนทำงานแทน Agent

งานแบบไหนควรทำเป็น Workflow

Workflow มีต้นทุน — เขียนไว้แล้วไม่ได้ใช้ ก็เสียเวลาเปล่าเหมือนกัน

สัญญาณว่าควรทำ

- ✓ มีหลายขั้นที่ต้องทำตามลำดับ
- ✓ ชั้นกลางมีโอกาสผิดโดยที่เรายังไม่เห็น
- ✓ งานใช้เวลานานจนไม่อยากรันซ้ำทั้งหมด
- ✓ มีจุดที่ต้องให้คนอนุมัติระหว่างทาง
- ✓ ต้องส่งต่อให้คนอื่นหรือ Agent ตัวอื่นทำต่อ

สัญญาณว่ายังไม่ต้อง

- ✗ สั่งครั้งเดียวแล้วจบได้
- ✗ ผิดแล้วรู้ทันทีจากผลลัพธ์
- ✗ รันใหม่ทั้งงานใช้เวลาไม่ถึงนาที
- ✗ ไม่มีอะไรที่ย้อนกลับไม่ได้เลย
- ✗ ทำเองคนเดียวจบในหัวได้

เริ่มจากงานที่เคยรันแล้วพังกลางทางมาก่อน — นั่นคือ Workflow แรกที่คุ้มที่สุด

เขียนขั้นตอนให้ตรวจได้

เขียนแบบนี้ Agent เค้าเอง

“ค้นข้อมูลให้ครบ”



เขียนแบบนี้ตรวจได้

ค้นอย่างน้อย 5 แหล่ง แล้วแนบลิงก์ทุกแหล่งในไฟล์ research.md

“วิเคราะห์ให้ดี”



เทียบ 3 คู่แข่งในตารางเดียว ช่องละไม่เกิน 1 ประโยค

“ถ้าติดปัญหาให้แก้”



ถ้าเปิดลิงก์ไม่ได้ ให้ข้ามและบันทึกไว้ท้ายไฟล์ ห้ามเดาเนื้อหา

“ตรวจงานก่อนส่ง”



สุ่มตรวจข้ออ้าง 2 ข้อย้อนกลับไปหาแหล่ง แล้วรายงานผลการตรวจ

“สรุปส่งมาให้หน่อย”



เขียน summary.md ยาวไม่เกิน 1 หน้า ปิดท้ายด้วยสิ่งที่ยังไม่รู้

กฎเดียวที่ใช้ตัดสิน

อ่านขั้นนั้นแล้วบอกได้ไหมว่า Agent “ทำตาม” หรือ “ไม่ทำตาม” — ถ้าเถียงกันได้ แปลว่ายังไม่ชัดพอ

อาการของ Workflow ที่พัง

สื่แบบนี้เจอเกือบทุกคนในรอบแรก — รู้ล่วงหน้าแล้วแก้ได้ในไม่กี่วินาที

อาการ

Agent ข้ามขั้นแล้วไปตอบตอนจบเลย

ทางแก้

สั่งให้ส่งผลลัพธ์ของแต่ละขั้นเป็นไฟล์ แล้วรอ ok ก่อนเดินขั้นถัดไป

อาการ

รันไปเรื่อยจน Context เต็มแล้วลืมโจทย

ทางแก้

ให้เขียนผลแต่ละขั้นลงไฟล์ แล้วขั้นถัดไปอ่านจากไฟล์ ไม่ใช่จากแชท

อาการ

ขั้นที่ลัมแล้ววนลองวิธีเดิมไม่จบ

ทางแก้

กำหนดจำนวนครั้งไว้ในโจทย และบังคับว่ารอบสองต้องเปลี่ยนวิธี

อาการ

ผลออกมาสวยแต่ขั้นกลางไม่ได้ทำจริง

ทางแก้

ขอ trace ว่าแต่ละขั้นเรียกเครื่องมืออะไร ได้ไฟล์อะไร แล้วเปิดดูไฟล์เอง

ทุกข้อแก้ด้วยหลักเดียวกัน — ทำให้แต่ละขั้นทิ้งหลักฐานไว้ข้างนอกแชท

02

Build Agent Workflow

Workshop 7 · 20:30 – 21:15

โจทย์ของ Workshop 7 · เลือกหนึ่งเส้น

ทุกเส้นเดินครบเจ็ดขั้นเหมือนกัน และจบที่ไฟล์ผลงานหนึ่งไฟล์เหมือนกัน

เส้น A

งานวิจัยและสรุป

เรื่องที่ต้องหาข้อมูลจากหลายแหล่งแล้วสรุปให้คนอื่นอ่าน

- 1 ค้นอย่างน้อย 5 แหล่ง
- 2 เทียบและตัดที่ไม่เกี่ยว
- 3 สรุปหนึ่งหน้าพร้อมที่มา

เส้น B

งานข้อมูลในเครื่อง

ไฟล์งานจริงที่มีอยู่แล้ว — csv, log, เอกสารเก่าของทีม

- 1 สืบหาโครงสร้างก่อน
- 2 ตั้งคำถามที่ตอบได้จริง
- 3 รายงานพร้อมตัวเลขอ้างอิง

เส้น C

งานประจำหลายขั้น

งานที่เราทำเองทุกสัปดาห์แล้วใช้เวลานาน ให้ Agent เดินแทนหนึ่งรอบ

- 1 เขียนขั้นตอนที่ทำจริง
- 2 ให้ Agent เดินตามทีละขั้น
- 3 จดจุดที่มันหลุดจากขั้น

เลือกภายใน 2 นาทีแล้วเริ่มเลย — โจทย์ที่เราสนใจจริงจะเดินเร็วกว่าโจทย์ที่ฟังดูดี

Prompt ตั้งต้น · คัดลอกไปใช้ได้เลย

เปลี่ยนเฉพาะข้อความในวงเล็บ ที่เหลือปล่อยไว้ตามนี้

ทำงานอยู่ในโฟลเดอร์ day4-workshop เท่านั้น

โจทย์ใหญ่: [เขียนโจทย์ของเส้นที่เลือก 1-2 บรรทัด]

ให้ทำตาม Workflow นี้ ทีละขั้น อย่าข้าม

1. เข้าใจโจทย์ — ถามกลับถ้ามีอะไรกำกวม
2. วางแผน — แยกเป็น 4-7 ขั้น แล้วรอผมตอบ ok
3. ค้น/อ่าน — ใช้เครื่องมือจริง บันทึกแหล่งที่มาทุกครั้ง
4. วิเคราะห์ — ตัดสิ่งที่ไม่เกี่ยวกับโจทย์ออก
5. ลงมือ — เขียนผลลงไฟล์ในโฟลเดอร์นี้
6. ตรวจสอบ — สุ่มตรวจข้ออ้าง 2 ข้อย้อนกลับไปหาแหล่ง
7. ส่งผล — สรุปสิ่งที่ทำ ไฟล์ที่ได้ และสิ่งที่ยังไม่แน่ใจ

จบทุกขั้น ให้บอกว่าใช้เครื่องมืออะไรและได้ไฟล์อะไร

ถ้าขั้นไหนลَمْ ให้หยุดแล้วรายงาน อย่าเดาต่อ

Prompt ที่ดีคือสัญญาว่าจะส่งอะไรกลับในแต่ละขั้น ไม่ใช่คำอธิบายว่าอยากได้อะไรสวย ๆ

ทำไมต้องมีบรรทัดพวกนี้

ขอบเขต

กันไม่ให้ Agent เดินออกไปแตะไฟล์อื่นในเครื่อง

รอ ok ที่ขั้น 2

ได้เห็นแผนก่อนเสียเวลา 30 นาทีไปกับแผนที่ผิด

บันทึกแหล่งที่มา

ทำให้ขั้นตอนตรวจสอบมีของให้ตรวจจริง ไม่ใช่ตรวจความรู้สึก

เขียนลงไฟล์

ขั้นถัดไปอ่านจากไฟล์ได้ Context จึงไม่บวมจนลืมโจทย์

สุ่มตรวจ 2 ข้อ

เปลี่ยนจาก “ดูน่าเชื่อ” เป็น “ตรวจแล้วตรง”

ลَمْแล้วให้หยุด

กันการเดาต่อเจิบ ๆ ซึ่งเป็นความผิดพลาดที่แพงที่สุด

ลำดับงานของ Workshop 7



หนึ่งโจทย์ เดินครบเจ็ดขั้น โดยให้ Agent หยุดให้เราตรวจที่ขั้น 2 และขั้น 6 — ปลายทางคือไฟล์ผลงานที่เปิดอ่านได้จริง ไม่ใช่ข้อความในแชท

เวลาที่ให้ในแต่ละขั้น						
เข้าใจโจทย์	วางแผน	ค้นและอ่าน	วิเคราะห์	ลงมือทำ	ตรวจและส่ง	
3'	7'	10'	7'	10'	8'	

เดินทีละขั้น · ขั้นที่ I ถึง 3

O1	เข้าใจโจทย์ · 3 นาที สิ่งที่เราพิมพ์	วางโจทย์ใหญ่ แล้วให้ Agent ถามกลับก่อนว่ามีอะไร กำกวม ต้องเห็น · คำถามกลับที่ตรงจุด ไม่ใช่ “รับทราบครับ เริ่มเลยนะ ครับ”	ถ้ามันไม่ถามอะไรเลย แปลว่ามันกำลังจะเดา
O2	วางแผน · 7 นาที สิ่งที่เราพิมพ์	ให้แตกเป็น 4-7 ขั้น พร้อมบอกว่าแต่ละขั้นจะได้ ไฟล์อะไร ต้องเห็น · แผนเป็นข้อ ๆ ที่ระบุผลลัพธ์ของแต่ละขั้นไว้ชัด	ขั้นไหนไม่มีผลลัพธ์ ให้สังวบรวมกับขั้นข้าง ๆ
O3	ค้นและอ่าน · 10 นาที สิ่งที่เราพิมพ์	ให้ใช้เครื่องมือจริง แล้วเขียนผลลง research.md พร้อมแหล่งที่มา ต้องเห็น · ไฟล์จริงในโฟลเดอร์ ที่มีลิงก์หรือชื่อไฟล์กำกับทุก ข้อ	เจอสรุปที่ไม่มีที่มา ให้ถามกลับว่า “อันนี้มาจากไหน”

จับเวลาจริง — เกินเวลาให้ตัดขอบเขตโจทย์ลง อย่ายืดเวลา เพราะยังเหลืออีกสี่ขั้น

เดินทีละขั้น · ขั้นที่ 4 ถึง 7

04	วิเคราะห์ · 7 นาที สิ่งที่เราพิมพ์	ให้เทียบข้อมูลที่ได้ แล้วตัดสิ่งที่ไม่ตอบโจทย์ทิ้ง ต้องเห็น · สิ่งที่ถูกตัดออก พร้อมเหตุผลสั้น ๆ ว่าตัดเพราะอะไร	ถ้าไม่มีอะไรถูกตัดเลย แปลว่ายังไม่ได้วิเคราะห์
05	ลงมือทำ · 10 นาที สิ่งที่เราพิมพ์	สั่งให้เขียนผลงานลงไฟล์ แล้วเปิดไฟล์นั้นอ่านกลับมาให้เราดู ต้องเห็น · ไฟล์จริงในโฟลเดอร์ ไม่ใช่ข้อความยาว ๆ ในแชท	ให้ list โฟลเดอร์ยืนยันอีกรอบ กันเคลส “บอกว่าเขียนแล้ว”
06	ตรวจและส่ง · 8 นาที สิ่งที่เราพิมพ์	ให้สุ่มตรวจข้ออ้าง 2 ข้อย้อนกลับไปหาแหล่ง แล้วรายงานผล ต้องเห็น · ผลการตรวจที่บอกว่าข้อไหนตรง ข้อไหนยังไม่แน่ใจ	ขั้นที่ 7 คือให้มันสรุปทั้ง Workflow ไว้ใช้ซ้ำพุงนี้

เหลือ 5 นาทีสุดท้ายไว้แชร์ผล — เตรียมตอบสั้น ๆ ว่า “ขั้นไหนที่ Agent หลุดมากที่สุด”

สิ่งที่ต้องได้จาก Workshop 7

ตารางนี้คือเกณฑ์ตรวจ — ช่องขวาคือสิ่งที่ต้องชี้ให้ได้ ไม่ใช่สิ่งที่รู้สึกว่าได้ทำแล้ว

ขั้นตอน	สิ่งที่ AGENT ทำ	สิ่งที่ต้องเห็น
เข้าใจโจทย์	ถามกลับจุดที่กำกวม	คำถามที่ตรงกับโจทย์จริง
วางแผน	แตกเป็น 4-7 ขั้นตอนพร้อมผลลัพธ์	แผนที่ระบุไฟล์ของแต่ละขั้น
ค้นและอ่าน	ใช้เครื่องมือจริงแล้วบันทึกที่มา	ไฟล์ research พร้อมแหล่งอ้างอิง
ลงมือทำ	เขียนผลงานลง Workspace	ไฟล์จริงในโฟลเดอร์ของตัวเอง
ตรวจสอบ	สุ่มตรวจย้อนกลับแล้วรายงาน	ผลตรวจที่แยกข้อผ่านกับข้อค้าง

ทั้งห้าแถวนี้ใช้เวลารวม 38 นาที — ที่เหลือเพื่อไว้ให้ตรวจและแก้

เช็ีกก่อนบอกว่าเสร็จ

ดีกัให้ครบทั้งหกข้อ — ขาดข้อเดียวแปลว่ายังเป็นแค่ข้อความที่ฟังดูดี

- ☐ มีไฟล์ผลงานอยู่จริงในโฟลเดอร์ และเราเปิดอ่านแล้ว
- ☐ แต่ละขั้นทั้งหลักฐานไว้ — ไฟล์ ลิงก์ หรือ trace ของเครื่องมือ
- ☐ Agent หยุดให้เราตรวจสอบอย่างน้อยหนึ่งครั้งก่อนลงมือ
- ☐ เราสุ่มตรวจสอบย้อนกลับแล้วอย่างน้อยสองข้อ และมันตรง
- ☐ มีหัวข้อ “สิ่งที่ยังไม่แน่ใจ” และมันไม่ว่างเปล่า
- ☐ มีลำดับขั้นที่เอาไปรันซ้ำพรุ่งนี้ได้โดยไม่ต้องนึกเอง

ข้อที่คนพลาดมากที่สุดคือข้อ 3

เราชอบตอบ ok ผ่าน ๆ เพราะอยากให้มันเดินต่อ — แต่ขั้นวางแผนคือจุดที่แก้ถูกที่สุดในทั้ง Workflow

ติดตรงนี้ · ทำแบบนี้

เจอบ่อยที่สุดสี่แบบระหว่าง Workshop — แก้ได้ในไม่กี่วินาที

! มั่นวางแผนมา 15 ชั้น จนไม่รู้จะเริ่มตรงไหน	→ “รวบให้เหลือ 5 ชั้น ชั้นละมีไฟล์ผลลัพธ์หนึ่งไฟล์” แล้วให้เสนอใหม่
! เดินชั้น 3 แล้วกระโดดไปเขียนสรุปเลย	→ “กลับไปทำชั้น 4 ก่อน แล้วบอกผมว่าตัดอะไรออกบ้าง” — อย่าปล่อยผ่าน
! ค้นข้อมูลแล้วได้แต่ของทั่วไป ไม่ตรงโจทย์	→ ระบุขอบเขตให้แคบ — ช่วงเวลา ประเทศ ชื่อสินค้า แล้วสั่งให้ค้นซ้ำด้วยคำที่แคบลง
! งานยาวจนมันเริ่มสับสนตอนที่ตกลงกันไว้	→ “สรุปสถานะและชั้นที่ทำไปแล้ว 5 บรรทัด” แล้วเปิด Session ใหม่ด้วยสรุปนั้น + ไฟล์ที่มี

ทุกข้อคือเรื่องเดียวกัน — บังคับให้ Workflow ที่หลักฐานไว้ทุกชั้น แล้วปัญหาจะเห็นตั้งแต่ต้น

เสร็จก่อนเวลา · ลองต่ออีกสามข้อ

ทั้งสามข้อคือสิ่งที่จะได้ใช้จริงใน Workshop 8 ตอนแบ่งงานให้หลาย Agent

ให้มันตรวจงานตัวเอง จะเห็นว่ามันหาจุดอ่อนของตัวเองเจอ ถ้าเราสั่งให้หา	“อ่านไฟล์ที่เพิ่งเขียน แล้วหาข้ออ้างที่ยังไม่มีหลักฐานรองรับ”
ให้มันจำลองว่าขั้นหนึ่งล้ม ได้เห็นว่า Error Handling ที่เขียนไว้ใช้ได้จริงหรือเปล่า	“สมมติว่าขั้นที่ 3 เปิดลิงก์ไม่ได้เลยสักอัน คุณจะทำอะไรต่อ”
ให้มันเขียน Workflow เป็นไฟล์ ไฟล์นี้คือวัตถุดิบตั้งต้นของ Workshop 8 และของวันที่ 5	“เขียนขั้นตอนทั้งหมดที่เพิ่งทำลง workflow.md ให้คนอื่นรันตามได้”

ยังไม่เสร็จก็ไม่ใช่ไร — เอาเท่าที่ได้ไปต่อใน Workshop 8 ได้เลย

03

Sub-agents & Multi-Agent

Session 8 · 21:15 – 21:35

Agent เดียว หรือหลาย Agent

ONE AGENT

ตัวเดียวทำครบทุกอย่าง

เร็วและง่ายที่สุดเมื่องานอยู่ในเรื่องเดียวกัน แต่พอยาวขึ้น Context จะบวม และมันจะเริ่มตรวจงานตัวเองแบบเข้าข้างตัวเอง

AGENT TEAM

แบ่งบทบาทแล้วส่งงานกัน

แต่ละตัวถือ Context เฉพาะงานของตัวเอง ตัวตรวจไม่ใช่ตัวเขียน จุดอ่อนจึงถูกจับได้ แลกมาด้วยความซับซ้อนของการส่งงาน

เส้นแบ่งที่ใช้ตัดสินจริง		
ความยาวของงาน	จบในไม่กี่สัปดาห์	ยาวจนจำดันทางไม่ไหว
ความต่างของทักษะ	ใช้วิธีคิดแบบเดียวตลอด	ค้น เขียน ตรวจ คนละแบบกัน
ใครตรวจใคร	ตรวจงานตัวเอง	คนละตัวกับคนทำ

เมื่อไหร่ควรแบ่งงาน

หลาย Agent ไม่ได้แปลว่าดีกว่า — แปลว่าแพงกว่าและช้ากว่า ถ้าไม่มีเหตุผลรองรับ

สัญญาณว่าควรแบ่ง

- ✓ งานยาวจน Context เต็มก่อนจบ
- ✓ ต้องมีคนตรวจที่ไม่ใช่คนทำ
- ✓ แต่ละส่วนใช้เครื่องมือคนละชุดกัน
- ✓ บางส่วนทำขนานกันได้จริง
- ✓ อยากเปลี่ยนแค่บางบทบาทโดยไม่รื้อทั้งระบบ

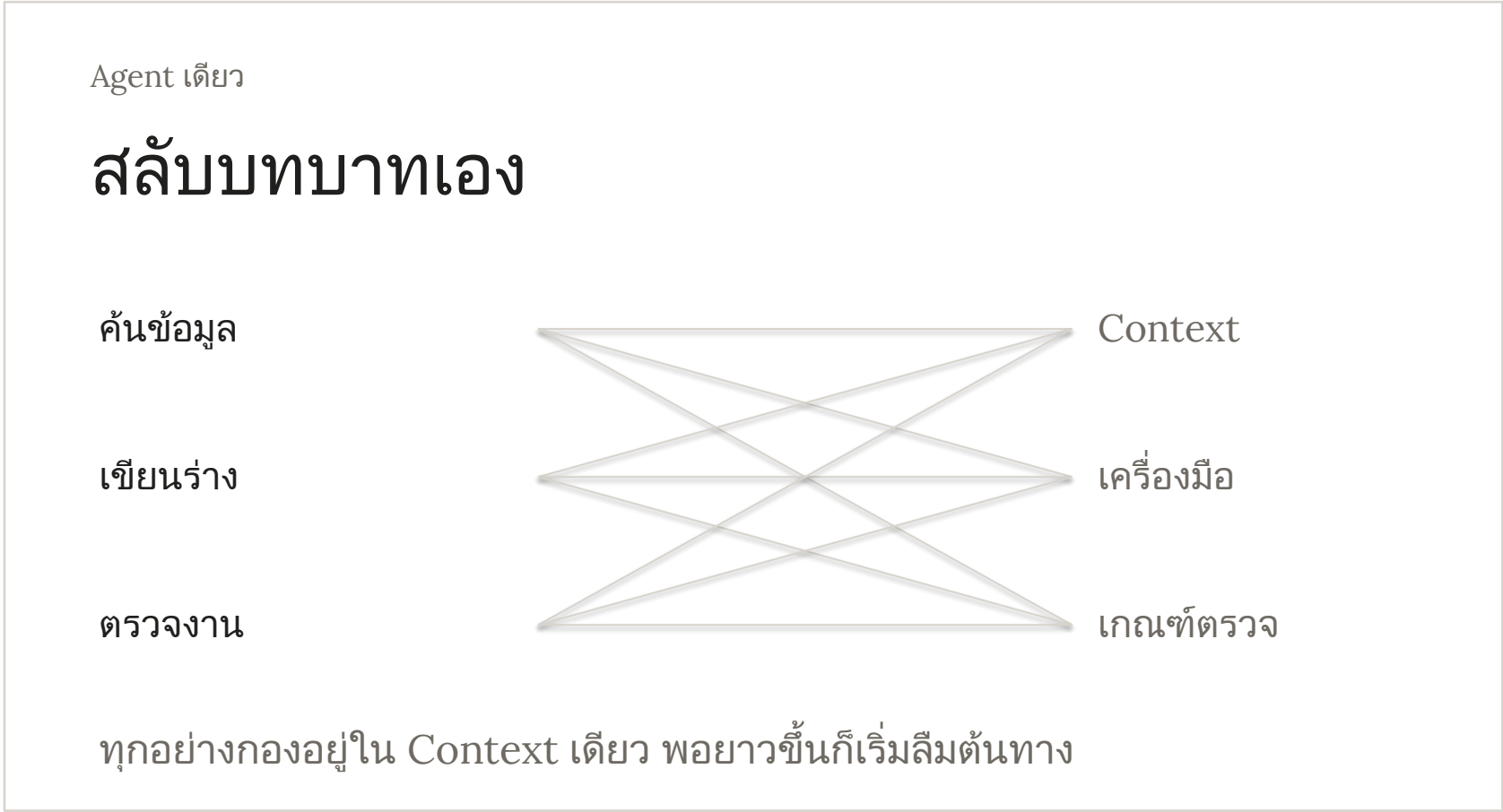
สัญญาณว่ายังไม่ต้อง

- ✗ งานจบได้ในรอบเดียว
- ✗ ผลลัพธ์ตรวจสอบได้จากไฟล์
- ✗ ใช้เครื่องมือชุดเดียวทั้งงาน
- ✗ ทุกขั้นต้องรอขั้นก่อนหน้า
- ✗ ยังไม่รู้เลยว่าขั้นตอนที่ดีคืออะไร

ถ้ายังตอบไม่ได้ว่า “ตัวไหนรับอะไร ส่งอะไร” ให้กลับไปทำ Workflow ตัวเดียวให้หนึ่งก่อน

งานเดินอย่างไรเมื่อมี Orchestrator

ปัญหาไม่ใช่ว่า Agent ตัวเดียวทำไม่ได้ — แต่มันต้องสลับบทบาทในหัวตัวเองตลอดเวลา



เหมือนทีมงานจริง — ไม่ได้เก่งขึ้นเพราะคนเยอะ แต่เพราะแต่ละคนรับผิดชอบเรื่องเดียวและมีคนตรวจ

สามส่วนของการส่งงานระหว่าง Agent

ROLE

แต่ละตัวเป็นใคร

หน้าที่เดียวที่ชัด เครื่องมือเท่าที่จำเป็น
และข้อห้ามที่เขียนไว้ตรง ๆ

คำถามที่ต้องตอบให้ได้

ตัวนี้ทำอะไรได้ · อะไรที่ห้ามทำเด็ดขาด

HANDOFF

ส่งอะไรให้กัน

รูปแบบข้อมูลที่ตกลงกันไว้ล่วงหน้า ปกติคือ
ไฟล์ในโฟลเดอร์ ไม่ใช่ข้อความลอย ๆ

คำถามที่ต้องตอบให้ได้

ไฟล์ชื่ออะไร · มีหัวข้ออะไรบ้าง · ใครเขียน

GATE

ผ่านเมื่อไหร่

เกณฑ์ที่บอกงานของตัวก่อนหน้าใช้ได้
แล้ว ถ้าไม่ผ่านให้ตีกลับพร้อมเหตุผล

คำถามที่ต้องตอบให้ได้

ตีกลับเมื่อไหร่ · ตีกลับแล้วใครแก้

สามส่วนนี้คือสิ่งเดียวที่ทำให้ทีม Agent ต่างจากการเปิดหลายแชทแล้วคัดลอกไปมาเอง

ลำดับการออกแบบที่เร็วที่สุด

I เขียน Handoff ก่อน

รู้ว่าส่งอะไรกัน แล้วบทบาทจะชัดเจนเอง

2 แล้วค่อยเขียน Role

ใครต้องผลิตไฟล์นั้น คนนั้นคือบทบาทที่ต้องมี

3 ปิดท้ายด้วย Gate

เหลือแค่ว่าไฟล์แบบไหนเรียกว่าใช้ได้

ห้าบทบาทของทีม AI

เริ่มจากสามตัวก่อนก็พอ — Researcher, Writer, Reviewer ครอบคลุมงานส่วนใหญ่แล้ว

Orchestrator	รับโจทย์และคุมลำดับ	แตกงาน กระจาย รวมผล และตัดสินใจว่าจะดีกลับไหม
Researcher	หาและอ่านข้อมูล	ส่งออกเป็นข้อเท็จจริงพร้อมแหล่งที่มา ไม่ตีความ
Analyst	เทียบและตัดสินใจ	รับข้อเท็จจริง ส่งคืนข้อสรุปพร้อมเหตุผลและสิ่งที่ตัดทิ้ง
Writer	เรียบเรียงให้คนอ่าน	รับข้อสรุป ส่งคืนร่างตามรูปแบบที่ตกลงไว้
Reviewer	ตรวจและดีกลับ	รับร่าง ส่งคืนรายการจุดแก้ พร้อมบอกว่าข้อไหนบล็อกร่าง
คุณ (มนุษย์)	อนุมัติและรับผิดชอบ	ดูผลตรวจแล้วตัดสินใจว่าจะปล่อยงานออกหรือให้แก้

บทบาทที่คนมักลืมใส่คือ Reviewer — และมันคือตัวที่เปลี่ยนคุณภาพมากที่สุด

สี่กับดักของทีม AI

ทีมที่ออกแบบไม่ดี ให้ผลแย่กว่า Agent ตัวเดียว และแพงกว่าด้วย

อาการ

ทุกตัวได้ Context ก้อนเดียวกันทั้งหมด

ทางแก้

ส่งเฉพาะไฟล์ที่ตัวนั้นต้องใช้ — แบ่งบทบาทแล้วแต่ไม่แบ่งข้อมูล
ก็ไม่ได้อะไรเพิ่ม

อาการ

Reviewer ผ่านทุกอย่างโดยไม่ค้านอะไรเลย

ทางแก้

ให้เกณฑ์ตรวจเป็นข้อ ๆ และบังคับให้รายงานข้อที่ไม่ผ่านอย่างน้อยหนึ่งข้อ

อาการ

ติ๊กกลับไปมาไม่จบ งานวนอยู่ในทีม

ทางแก้

กำหนดรอบติ๊กกลับสูงสุด 2 ครั้ง ครบแล้วให้ส่งขึ้นมาให้คนตัดสินใจ

อาการ

Orchestrator ลงไปทำงานเองแทนที่จะกระจาย

ทางแก้

ตัดเครื่องมือของ Orchestrator ให้เหลือแค่การอ่านไฟล์และมอบหมายงาน

ทุกกับดักมาจากที่เดียวกัน — บทบาทเขียนไว้สวย แต่ข้อมูลกับสิทธิ์ยังไม่ได้แยกจริง

ออกแบบการส่งงานให้ไม่หลุด

จุดที่ทีม Agent พังบ่อยที่สุดไม่ใช่ตัว Agent — แต่เป็นช่องว่างระหว่างตัวหนึ่งกับอีกตัว

01	ส่งกันด้วยไฟล์ ไม่ใช่ข้อความในแชท	ไฟล์เปิดย้อนดูได้ แก้ได้ และไม่หายไปพร้อม Context
02	ตกลงหัวข้อของไฟล์ไว้ล่วงหน้า	ตัวรับรู้ว่าจะเจออะไร จึงไม่ต้องเดาโครงสร้างทุกครั้ง
03	ทุกไฟล์ต้องบอกว่าใครเขียนและอ้างอิงอะไร	พอผลลัพธ์ผิด เราตามได้ทันทีว่าหลุดที่บทบาทไหน
04	เขียนเงื่อนไขติกลับไว้ในไฟล์เดียวกัน	Reviewer ไม่ต้องอธิบายยาว แค่ชี้ข้อที่ไม่ผ่านตามเกณฑ์ที่ตกลงไว้

ถ้าออกแบบ Handoff ดีพอ เราจะเปลี่ยน Agent ตัวไหนก็ได้โดยไม่ต้องรื้อทั้งระบบ

04

AI Team Architecture

Workshop 8 · 21:45 – 22:00

Workshop 8 · AI Team Architecture

21:45 – 22:00 น. · เอา Workflow จาก Workshop 7 มาแบ่งเป็นบทบาท แล้วเขียน Architecture ไว้ใช้ต่อวันที่ 5

01	เลือกงานจาก Workshop 7 แล้วชี้ว่าขั้นไหนควรเป็นคนละบทบาท	3 นาที	ขั้นที่ใช้ทักษะต่างกันชัด ๆ คือรอยต่อที่ควรตัด
02	กำหนด Role, Input และ Output ของแต่ละตัว	6 นาที	เขียน Output ก่อน แล้ว Input ของตัวถัดไปจะชัดเอง
03	วาดเส้นส่งงานและจุดติกลับให้ครบทุกเส้น	3 นาที	เส้นที่ไม่มีไฟล์รองรับ คือเส้นที่จะหลุดตอนรันจริง
04	ลองรันหนึ่งรอบด้วยการสวมบทบาททีละตัว	3 นาที	ยังไม่ต้องตั้งระบบจริง ขอให้รู้ว่าเส้นไหนขาด

จบ 15 นาทีแล้วต้องมีไฟล์ team.md หนึ่งไฟล์ ที่บอกได้ว่าใครทำอะไร รับอะไร ส่งอะไร

เลือกโจทย์ทีมใน 2 นาที

ตอบสามข้อนี้ในหัว ถ้าตอบได้ครบแปลว่าโจทย์ใช้ได้ เริ่มเขียนได้เลย

O1

ในงานของเรา ขึ้นโหนดที่ “คนทำ” กับ “คนตรวจ” ควรเป็นคนละคน

ตัวอย่าง · เขียนข้อเสนอแล้วตรวจตัวเลข · ร่างอีเมลแล้วตรวจน้ำเสียง · สรุปข้อมูลแล้วตรวจที่มา

O2

มีขึ้นโหนดที่ทำขนานกันได้จริงบ้าง

ถ้าค้นข้อมูลสามแหล่งพร้อมกันได้ นั่นคือ Sub-agent สามตัวที่คุ้มที่สุด

O3

ถ้าตัวใดตัวหนึ่งทำพลาด เราจะรู้ได้จากอะไร

ถ้าตอบไม่ได้ แปลว่ายังไม่มี Gate — ให้เขียนเกณฑ์ตรวจก่อนเริ่ม

ต้น 2 นาทีแล้วยังไม่ได้ ให้ใช้โครง Researcher → Writer → Reviewer กับงานของ Workshop 7 ไปเลย

เทมเพลต · เติมช่องว่างแล้วใช้ได้เลย

เขียนให้จบใน 6 นาที แล้วค่อยแก้ — สั้นเกินแก้ง่ายกว่ายาวเกิน

ORCHESTRATOR	RESEARCHER	WRITER	REVIEWER
โจทย์ใหญ่: _____	รับ: โจทย์ + ขอบเขต	รับ: research.md	รับ: draft.md + research.md
แตกเป็น _____ บทบาท	ใช้: _____	ใช้: Skill _____	เกณฑ์: _____
ติ๊กกลับได้ไม่เกิน 2 รอบ	ส่ง: research.md + แหล่งที่มา	ส่ง: draft.md ตามรูปแบบ _____	ส่ง: review.md ระบุข้อที่บล็อกลอก
		ห้าม: เพิ่มข้อมูลที่ไม่มีในไฟล์	

ตัวอย่างที่เติมแล้ว

โจทย์: สรุปคู่แข่ง 3 ราย · Researcher ใช้ Search + File · Writer ใช้ Skill สรุปหนึ่งหน้า · Reviewer ตรวจสอบว่าทุกตัวเลขมีที่มา

ช่องว่างทุกช่องต้องเติมได้ด้วยคำที่ตรวจได้ — ถ้าเติมแล้วยังเถียงกันได้ว่าทำตามหรือยัง ให้เขียนใหม่

ทดสอบ Architecture ของตัวเอง · สามรอบ

เขียนเสร็จยังไม่นับ — ผ่านสามรอบนี้แล้วค่อยเรียกว่าออกแบบเสร็จ

รอบที่ 1

เดินตามเส้นปกติ

สวมบทบาททีละตัว แล้วส่งไฟล์ต่อกันจริงหนึ่งรอบ

✓ ทุกตัวได้ไฟล์ที่ต้องใช้ครบโดยไม่ต้องถามเพิ่ม

✗ ถ้าต้องอธิบายปากเปล่า แปลว่า Handoff ยังไม่ครบ

รอบที่ 2

ให้ Reviewer ตีกลับ

จงใจส่งร่างที่ขาดที่นำไปหนึ่งข้อ แล้วดูว่าระบบทำอะไรต่อ

✓ ตีกลับถูกตัว พร้อมบอกว่าต้องแก้อะไร

✗ ถ้าผ่านฉลุย แปลว่า Gate ยังไม่มีเกณฑ์จริง

รอบที่ 3

ตัดตัวหนึ่งออก

ลองเอา Analyst ออก แล้วดูว่างานยังเดินได้ไหม

✓ งานยังเดินได้ แปลว่าเริ่มด้วยทีมเล็กกว่านี้ก็พอ

✗ ถ้าพังทันที แปลว่าตัวนั้นจำเป็นจริง เก็บไว้

รอบที่ 3 คือรอบที่ช่วยประหยัดที่สุด — ทีมเล็กที่เดินได้ ดีกว่าทีมใหญ่ที่ยังไม่เคยรัน

สิ่งที่ต้องได้จาก Workshop 8

เขียนให้ครบทั้งตาราง — ช่องไหนเว้นว่าง คือเส้นที่จะหลุดตอนเอาไปใช้จริงวันที่ 5

บทบาท	รับอะไรเข้ามา	ส่งอะไรออกไป
Orchestrator	โจทย์ใหญ่จากเรา	งานที่แตกแล้ว และผลรวมตอนจบ
Researcher	โจทย์และขอบเขตที่กำหนด	ข้อเท็จจริงพร้อมแหล่งที่มา
Writer	ข้อเท็จจริงที่ผ่านการคัดแล้ว	รายงานตามรูปแบบที่ตกลงไว้
Reviewer	รายงานและข้อมูลต้นทาง	รายการจุดแก้ และข้อที่บล็อกรายการส่ง
คุณ (มนุษย์)	ร่างที่ผ่านการตรวจแล้ว	การอนุมัติ หรือคำสั่งให้แก้

ไฟล์ team.md นี้คือจุดเริ่มต้นของ AI Employee ที่จะประกอบในคืนสุดท้าย

ทีม AI ใช้จริงหน้าตาเป็นแบบไหน

สามตัวอย่างนี้เป็นโครงที่เอาไปปรับกับงานของตัวเองได้เลย

งานเนื้อหา

- Researcher หาข้อมูลและที่มา
- Writer ร่างตามรูปแบบเดิม
- Reviewer ตรวจสอบข้อเท็จจริง

งานวิเคราะห์ข้อมูล

- ดึงข้อมูลจากหลายแหล่งพร้อมกัน
- Analyst รวมและเทียบ
- Reviewer ตรวจสอบสูตรและสมมติฐาน

งานบริการลูกค้า

- อ่านเรื่องและจัดประเภท
- ร่างคำตอบตาม Playbook
- คนอนุมัติก่อนส่งออกจริง

ข้อควรระวังที่ต้องพูดถึงเสมอ

ยิ่งมี Agent มาก ยิ่งต้องมีจุดที่คนอนุมัติชัดขึ้น ไม่ใช่ปล่อยลง

เริ่มจากสามบทบาทและให้คนอนุมัติทุกครั้ง แล้วค่อยลดการอนุมัติเมื่อเห็นผลตรวจสอบย้อนหลังว่านิ่งจริง

สั่ง Orchestrator อย่างไรให้กระจายงานถูก

Orchestrator ที่ไม่มีเกณฑ์ จะลงไปทำงานเองทุกครั้ง

เราพิมพ์แบบนี้

“ช่วยสรุปคู่แข่งสามรายให้หน่อย”

“ตรวจร่างนี้ให้ที”

“ทำรายงานเดือนนี้ให้เหมือนเดือนที่แล้ว”

Orchestrator ต้องแปลงเป็นแบบนี้

Researcher ×3 (ขนาน) → Analyst → Writer → Reviewer

Reviewer ตัวเดียว ไม่ต้องเรียกใครเพิ่ม

Researcher → Writer (ใช้ Skill เดิม) → Reviewer

วิธีทดสอบที่เร็วที่สุด

ให้ Orchestrator ตอบก่อนว่า “จะเรียกใครบ้าง และแต่ละตัวได้ไฟล์อะไร” — ถ้ามันตอบไม่ได้ อย่าเพิ่งให้เริ่มทำ

สรุปวันนี้

01

Agent วางแผนและแตกงานใหญ่เป็นขั้นที่ตรวจสอบได้

แผนต้องมี Goal · Steps · Done ที่เรียงกันไม่ได้

02

Workflow มีจุดหยุด จุดตรวจ และทางออกเมื่อพลาด

Error Handling · Retry · Verification · Human-in-the-loop

03

ทีม AI แบ่งบทบาทและส่งงานกันด้วยไฟล์

Role · Handoff · Gate — สามส่วนที่ทำให้ทีมไม่หลุด

NEXT

Build
Your AI
Employee

18 ก.ย. 2569 · 20:00 น.

รวม Memory, Tools, Skills,
Workflow และทีม Agent
เป็นพนักงาน AI หนึ่งคน